Lab 1 — Regular Expressions

Learning objectives

- Understand the basic principles of regular expressions.
- Match specific textual patterns using character classes, quantifiers and anchors.
- Extract structured information (dates, emails, URLs, numbers) from unstructured text.
- Replace and clean patterns using re.sub().
- Combine several patterns into a small information-extraction and text-anonymization pipeline.

Exercise 1 — Literal matching and counting

Scenario: You are given a short corpus made of three sentences mixing two topics.

```
import re

text = "NLP is useful. NLP is interesting. Machine learning is useful."

print(re.findall(r"NLP", text))
print(re.findall(r"useful", text))
```

Your task

1. Run the code above and note how many times each pattern is found.
2. Write a pattern that matches NLP only as a whole word (not as part of another word such as NLPs), using \b.
3. Write a function count_occurrences(pattern, text) that returns the number of matches, and use it to count how many times useful OR interesting appears (hint: use the | alternation operator).
4. Make the search case-insensitive and test it on a version of the text where NLP is sometimes written nlp or Nlp.

Questions

1. What does each pattern match, and how many occurrences are found?
2. What is the difference between searching for NLP and using the alternation pattern useful|interesting?
3. Why does \bNLP\b behave differently from NLP alone on a text containing the word NLPs?

> ***Challenge (going further):*** *Write a single regular expression that counts how many sentences mention at least one of the two topics (NLP or machine learning), and report the percentage of the three sentences that do.*

Exercise 2 — Character classes and a simple validator

```
text = "cat bat rat dog hot"

print(re.findall(r"[cb]at", text))
print(re.findall(r"[aeiou]", text))
```

Your task

1. Modify the pattern to find every word ending in at, regardless of the letter before it.

2. Write a pattern that matches any three-letter word made only of consonants (no a, e, i, o, u).
3. Using character classes and quantifiers, write a simple username validator: a valid username must be 4 to 12 characters long and contain only lowercase letters, digits and underscores. Test it on: 'ali_92', 'A', 'student.name', 'nlp_lab_2026'.
4. For every username above, print whether it is VALID or INVALID and, if invalid, give the reason (too short, forbidden character, etc.).

Questions

1. What does [cb] mean, and which words match [cb]at?
2. Why is a character class not enough to validate a minimum or maximum length? What quantifier syntax did you need?
3. Which of the four test usernames failed, and why exactly?

Exercise 3 — Digits, decimals and prices

```
text = "The course has 20 hours, 3 labs and 12 exercises."

print(re.findall(r"\d+", text))
```

Scenario: A shop receipt mixes plain quantities, decimal weights and prices with a currency symbol: "2 apples, 1.5 kg of rice, 3 juices at $2.50 each, total: $12.75 (-$1.00 discount)."

Your task

1. Modify the digit pattern so that it also matches decimal numbers such as 1.5 and 2.50.
2. Extend the pattern so that it correctly captures negative amounts such as -$1.00.
3. Write a pattern that extracts only the amounts that are preceded by a dollar sign, and store the numeric values (without the $) in a Python list of floats.
4. Compute the sum of all extracted dollar amounts using Python, and check it against the total you can compute by hand from the receipt.

Questions

1. What does \d represent, and what does + mean?
2. Why does the simple \d+ pattern fail to capture 1.5 or 2.50 as single numbers?
3. What extra character(s) did you need in your character class to handle the minus sign, and did you need to escape the dollar sign?

> ***Challenge (going further):*** *Write a regular expression (and a short Python snippet) that separates the quantities (whole numbers not attached to $) from the monetary amounts (numbers attached to $) in the receipt, and prints them as two separate lists.*

Exercise 4 — Dates and times in mixed formats

```
text = "The exam is on 16/09/2026 at 14:30."

print(re.findall(r"\d{2}/\d{2}/\d{4}", text))
print(re.findall(r"\d{2}:\d{2}", text))
```

Scenario: A poorly formatted schedule contains several date styles: "Lab 1: 16/09/2026, Lab 2: 2026-09-23, Lab 3: 30-09-2026, exam at 9:00, revision session at 14:30, office hours 09h30."

Your task

1. Extend your date pattern so that it matches both DD/MM/YYYY and DD-MM-YYYY formats using a single regular expression.
2. Write a second pattern that also matches the ISO format YYYY-MM-DD.
3. Write one combined pattern (using alternation or an optional/mixed separator class) that matches all three date formats in the schedule above in a single re.findall() call.
4. Write a pattern that matches both time formats 9:00 / 14:30 and 09h30, and explain what change was necessary.

Questions

1. Explain each part of the original two patterns (\d{2}, /, \d{4}, etc.).
2. What breaks when the day or month has only one digit, e.g. 6/9/2026? How would you adapt the pattern?
3. Why is it risky to validate a date only with a regular expression (think about 31/02/2026)?

Exercise 5 — Email addresses and a validity checker

```
text = "Contact ali@example.com or student.name@univ.dz."

pattern = r"[\w.-]+@[\w.-]+\.\w+"

print(re.findall(pattern, text))
```

Scenario: Here is a small mailing list to clean, containing valid and invalid entries: 'ali@example.com', 'student.name@univ.dz', 'a.b-c@sub.univ.dz', 'not-an-email', 'missing@domain', 'double@@example.com', '@no-local-part.com', 'ok.name+lab@example.co.uk'.

Your task

1. Apply the given pattern to every entry in the mailing list and note which ones are (incorrectly) accepted or rejected.
2. Improve the pattern so that it rejects 'double@@example.com', '@no-local-part.com' and 'missing@domain' (which has no dot after the @).
3. Test your improved pattern on 'ok.name+lab@example.co.uk'. Does it need the + character added to the local-part character class?
4. Write a function is_valid_email(address) built on your final pattern, apply it to the whole list, and print a report: number of valid addresses, number of invalid addresses, and the list of each.

Questions

1. Identify the different parts of the original pattern (local part, @, domain, TLD).
2. Which real-world email address, valid according to the official standard, does your pattern still fail to handle correctly? (Think about very short or unusual TLDs, or addresses without a dot at all.)
3. Why is writing a fully correct email-validating regular expression considered a notoriously hard problem?

Exercise 6 — URLs and domain extraction

```
text = "Visit https://www.univ.dz or http://example.org."

pattern = r"https?://[^\s]+"

print(re.findall(pattern, text))
```

Scenario: A paragraph contains several links: "See https://www.univ.dz/lab1?page=2, http://example.org, https://docs.python.org/3/library/re.html and ftp://old-server.com (not a web link)."

Your task

1. Apply the given pattern and check whether the trailing punctuation (a period after a URL, for example) is incorrectly captured as part of the link. Fix the pattern so trailing punctuation is excluded.
2. Write a second regular expression, applied only to the URLs already extracted, that captures just the domain name (e.g. www.univ.dz, example.org, docs.python.org).
3. Explain why the ftp:// link is correctly ignored by the https? pattern, and adapt the pattern so it also captures ftp:// links if needed.
4. Count how many of the extracted links use HTTPS versus HTTP.

Questions

1. What does https? mean, and what does [^\s]+ represent?
2. What problem did you observe with trailing punctuation, and how did you solve it?
3. Why can [^\s]+ still be considered a fairly naive way of matching a URL?

Exercise 7 — Substitution and a mini PII-redaction tool

```
text = "The course has 20 hours and 12 exercises."

result = re.sub(r"\d+", "<NUM>", text)

print(result)
```

Scenario: Before publishing student feedback online, personal information must be removed. A comment reads: "Contact me at ali.ben@example.com or call 0555 12 34 56 if you have questions about the 20-hour course. — Ali, 16/09/2026."

Your task

1. Modify the introductory example so that numbers are replaced with <NUMBER> instead of <NUM>.
2. Write a function anonymize(text) that, using re.sub() with the patterns built in the earlier exercises, replaces every email address with <EMAIL>, every phone number with <PHONE> and every date with <DATE>.
3. Apply anonymize () to the comment above and print the result.
4. Make sure your function leaves the word course and the number 20 (which is not personal information) unaffected — adapt your patterns if they are too aggressive.

Questions

1. What exactly has been replaced in the original example, and why is re.sub() well suited to this kind of task?
2. Which of the three patterns (email, phone, date) was hardest to get right, and why?
3. In what other real applications would this kind of automatic anonymisation be useful, and what are its limits?

> ***Challenge (going further):*** *Phone numbers can be written in several ways (0555123456, 0555 12 34 56, +213 555 12 34 56). Extend your phone-number pattern to cover at least two of these formats.*

Exercise 8 — RegEx analysis and limitations

Consider the following three patterns:

```
r"\d+"
r"[A-Z][a-z]+"
r"\b\w+\b"
```

Your task

1. For each pattern, explain in your own words what it matches, and give two example strings that match.
2. For each pattern, give one example where it fails or behaves unexpectedly (for instance, a capitalised acronym for pattern 2, or a hyphenated word for pattern 3).
3. Apply all three patterns to the sentence: "Dr. O'Brien met 3 students in New-York on 16/09/2026." and explain every unexpected result you observe.
4. Based on what you have seen in this exercise, list at least three linguistic phenomena (contractions, hyphenation, acronyms, multi-word names, etc.) that make regular expressions a fragile tool for general-purpose text segmentation.

> *No full Tokenizer is required in Lab 1 — the goal of this last exercise is to notice the limits of regular expressions before Lab 2 introduces dedicated Tokenization tools.*

Lab 1 mini-challenge — A small information-extraction report

Scenario: You are given the following short announcement: "Reminder: the NLP project (worth 12 points) is due on 30/09/2026 before 23:59. Send your report (max 2.5 MB) to nlp.lab@univ.dz or upload it at https://courses.univ.dz/submit. Late submissions lose -1.5 points per day. Contact +213 555 98 76 54 for questions."

Your task

1. Using the patterns, you built earlier in this lab, extract from the announcement: all dates, all times, all email addresses, all URLs, all phone numbers, and all numeric values (including decimals and negative numbers).
2. Store the results in a single Python dictionary, e.g. {"dates": [...], "times": [...], "emails": [...], "urls": [...], "phones": [...], "numbers": [...]}.
3. Print a short structured report listing each category and the values found.
4. Discuss which category was the easiest to extract reliably and which was the hardest, and why.